



Padrões de projeto

Este documento define as regras de organização, versionamento e escrita de código. É recomendado seguir este padrão para garantir um repositório limpo e um histórico rastreável.

1. Padrão para abertura de issues

Toda nova funcionalidade ou correção de bug deve ser iniciada através de uma issue no GitHub antes de qualquer linha de código ser escrita.

- As issues devem refletir exatamente o planejamento do nosso escopo.
- O título da issue deve ser direto e descritivo (ex: "Estruturação inicial do repositório e constantes de rede" ou "Criação do socket TCP passivo no servidor").
- O corpo da issue deve conter uma descrição do que precisa ser feito.
- O corpo da issue também deve conter uma sessão de "Tarefas" com um checklist (usando `[ ]` em Markdown) detalhando o passo a passo da implementação.

2. Nomenclatura de branches

Nunca faremos commits diretos na branch `main`. Todo desenvolvimento será feito em branches derivadas, nomeadas com a seguinte estrutura: `tipo/numero-da-issue-breve-descricao`.

Tipos permitidos:

- `feature/`: para novas funcionalidades mapeadas nas fases do projeto (ex: `feature/2-socket-tcp-servidor` para a issue 2).
- `bugfix/`: para correção de erros em funcionalidades que já estavam prontas.
- `docs/`: para atualizações nos arquivos do diretório `docs/` ou no `README.md`.

- `refactor/`: para reestruturação de código que não altera o comportamento final.

3. Padrão de commits

Adotaremos o padrão *conventional commits* para manter o histórico semântico. A estrutura da mensagem deve ser: `tipo(escopo): descrição concisa`.

- **Tipos:** `feat` (nova funcionalidade), `fix` (correção), `docs` (documentação), `chore` (manutenção, como organizar pastas).
- **Escopo:** deve refletir a organização de pastas da nossa arquitetura, utilize sempre `(servidor)`, `(cliente)` ou `(docs)`.
- A mensagem deve ser escrita no tempo verbal imperativo.

Exemplos práticos:

- `chore(docs): cria estrutura base de pastas e arquivo config.py` (Referente à issue 1).
- `feat(servidor): instancia socket TCP e adiciona modo listen na porta 5050` (Referente à issue 2).
- `feat(cliente): adiciona função escutar_servidor rodando em thread separada` (Referente às issues 11 e 12).
- `fix(cliente): adiciona os._exit() para fechar programa ao perder conexão` (Referente à issue 14).

4. Fluxo de pull requests e revisão

Para integrar o código de uma branch finalizada na `main`, é obrigatório abrir um PR.

- Nenhum membro pode aprovar o próprio PR.
- A mesclagem (merge) exige a aprovação de pelo menos 1 (um) membro da equipe.
- O PR deve referenciar a Issue que ele resolve na descrição (ex: `Closes #5` para vincular à Issue de implementação do fluxo echo).
- O revisor deve testar o código localmente executando os scripts `servidor.py` e `cliente/main.py` para garantir que não há bloqueios de porta ou falhas nas threads antes de aprovar.

5. Padrões de código

Como lidaremos intensamente com manipulação de rede e múltiplos fluxos de execução simultâneos, a clareza do código é importante.

- **Idioma:** seguiremos a nomenclatura em português para variáveis e funções customizadas (como `lidar_com_cliente`, `escutar_servidor`, `formatar_mensagem`) para diferenciar claramente dos métodos nativos em inglês da biblioteca `socket` e `threading` (como `.recv()`, `.sendall()`, `.start()`).
- **Formatação:** utilizaremos `snake_case` para variáveis e funções.
- **Tratamento de erros:** todas as rotinas de rede devem estar envolvidas em blocos `try...finally`.
- **Fechamento de sockets:** é obrigatório adicionar o fechamento explícito das conexões com `.close()` nos blocos `finally` para evitar portas presas (erros de "Address already in use") e o vazamento de recursos no sistema operacional.
- **Constantes:** valores fixos como endereços de IP de loopback (`127.0.0.1`), portas (`5050`) e tamanhos de buffer de rede (`1024`) devem ser importados de um arquivo de configuração central e escritos em `UPPER_CASE`.